GUIDE 48 · GOVERN

Sign in to the Console through your identity provider

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=48-sign-in-to-the-console-through-your-identity-provider.
This copy was made on 4 November 2026.

Time: 30 minutes. **You'll end up with:** administrators signing in to the Console through your identity provider (Keycloak, Entra ID, Okta, Google Workspace, …) - with its MFA and policies - and their role coming from the groups they're in there; someone leaving, or moving team, changes their access at their next sign-in, without anyone touching QueryAPIGate. Administrators who had tokens keep their names and history.

> **Stable** - Console sign-in through SSO is under the compatibility promise since 0.30 (what that means).

Every step below was run against a real Keycloak 26 (in Docker) and a real server, signing in through the Console in a real browser (Chromium) - the provider at `http://localhost:58080/realms/qag-sso-test` and the Console at `http://localhost:5094`, shown below as `https://idp.example.com/realms/acme` and `https://data.example.com`. Names you'll see: `alice` (in `qag-admins`), `bob` (in no group with a role) and `carol` (an existing administrator with a token, in `qag-owners`).

How it works

```text
Console --"Sign in with SSO"⟶ /auth/sso/login ⟶ your provider's sign-in (MFA, policies)
        <------------------- /auth/sso/callback ⟵ a code, exchanged by the server for an ID token
        session cookie, role from the person's groups
```

The **server** runs the sign-in (the authorization code flow with PKCE), as a confidential client with a secret; the browser never holds a token. It gets a session cookie - `HttpOnly`, `Secure`, `SameSite=Strict` - that only authenticates a request together with a CSRF token the Console sends; no other site can use it.

Step 1: At your provider - a client, and groups in the ID token

In Keycloak, in your realm:

1. **Clients** → **Create client**: OpenID Connect, client ID `queryapigate-console`. **Client authentication** on (a confidential client), **Standard flow** on, **Direct access grants** off.
2. **Valid redirect URIs**: `https://data.example.com/auth/sso/callback` - exactly the address people reach the Console on. **Valid post logout redirect URIs**: `https://data.example.com/*`.
3. **Advanced** → **Proof Key for Code Exchange Code Challenge Method**: `S256`.
4. **Credentials**: copy the client secret.
5. **Client scopes** → **queryapigate-console-dedicated** → **Add mapper** → **By configuration** → **Group Membership**: token claim name `groups`, **Full group path** off, **Add to ID token** on.
6. **Groups**: one per role you'll use - `qag-owners`, `qag-admins`, … - and put people in them.

Other providers name these differently; what QueryAPIGate needs is the same everywhere:

- **Microsoft Entra ID**: an app registration with a client secret and the redirect URI (type *Web*). Group claims are object IDs unless you emit names; app roles are often simpler - assign people to app roles and set `QUERYAPIGATE_SSO_GROUPS_CLAIM=roles`.
- **Okta**: an OIDC *Web Application* with the redirect URI, and a `groups` claim on its authorization server, filtered to your `qag-` groups and included in the ID token.
- **Google Workspace**: ID tokens carry no groups. Use `QUERYAPIGATE_SSO_DEFAULT_ROLE` with `QUERYAPIGATE_SSO_ALLOWED_DOMAINS` (below) - a Google client accepts any Google account, so the domain check is what keeps strangers out.

Check a real ID token: `iss` your provider, `aud` the client ID, `sub` there, `groups` (or your claim) holding the group names, and `email` with `email_verified: true`.

Step 2: Turn it on

```shell
QUERYAPIGATE_SSO_ISSUER=https://idp.example.com/realms/acme
QUERYAPIGATE_SSO_CLIENT_ID=queryapigate-console
QUERYAPIGATE_SSO_CLIENT_SECRET=...                      # from the provider
QUERYAPIGATE_SSO_REDIRECT_URL=https://data.example.com/auth/sso/callback
QUERYAPIGATE_SSO_ROLES=owner=qag-owners,admin=qag-admins
queryapigate serve
```

```text
Experimental: Console sign-in through SSO (QUERYAPIGATE_SSO_CLIENT_ID, QUERYAPIGATE_SSO_CLIENT_SECRET,
...) - may change in any minor release, ...
```

- `QUERYAPIGATE_SSO_ROLES` maps groups to roles - `owner`, `admin`, `developer`, `auditor`; a role may be listed more than once (`admin=qag-admins,admin=platform`). Someone in several gets the most capable.
- Everything must be `https://` (only `localhost` may be plain HTTP), and the Console must be reached over HTTPS: the session cookie is `Secure`. Put the server behind a reverse proxy with real TLS.
- Anything half-set stops startup, saying what's missing. **Settings → Console sign-in (SSO)** shows what's in use - the secret only as "configured".

Step 3: Sign in

The Console's sign-in page now offers **Sign in with SSO**, above the token field. alice clicks it, signs in at Keycloak, and lands back on the page she was on, *Signed in as alice · Admin via SSO*. Her administrator was created at that sign-in; her session, as the Console sees it:

```json
{"sso": true, "signed_in": true, "sso_only": false, "admin": "alice", "role": "admin", "email":
"alice@example.com",
 "csrf": "5sA4vM...", "expires_at": "2026-10-11 19:57:13"}
```

The cookie alone is not enough - the same request without the CSRF token gets `401`. In the audit log:

```json
{"actor": "alice", "via": "sso", "action": "create_admin", "target": "alice",
 "changes": {"role": "admin", "email": "alice@example.com", "issuer":
"https://idp.example.com/realms/acme"}}
{"actor": "alice", "via": "sso", "action": "sign_in", "target": "alice", "changes": {"role": "admin",
...}}
```

and every change she makes says `"via": "sso"`. **Sign out** ends the session here and at the provider.

Step 4: Someone with no role

bob signs in at Keycloak too - but he's in no group mapped to a role. Nothing is created; the Console says:

```text
Sign-in through SSO didn't go ahead: You signed in, but you're in no group that's given a role here -
ask an owner
to add you to one (sso_no_role)
```

```json
{"actor": "-", "via": "sso", "action": "sign_in_refused", "target": "bob@example.com",
 "changes": {"code": "sso_no_role", "reason": "You signed in, but you're in no group that's given a role
here - ..."}}
```

The role is decided **at every sign-in**: move alice from `qag-admins` to `qag-owners` and she's an owner from her next one; take her out of every mapped group and her next sign-in is refused.

Step 5: Administrators who had tokens

carol has been a developer with a personal token since before SSO. To keep her name and history, an owner marks her - **Administrators → Edit → Sign-in through SSO → Link at their next sign-in**, or:

```shell
curl -X PATCH https://data.example.com/api/v1/administrators/carol -H "X-API-Key: $TOKEN" \
  -H 'Content-Type: application/json' -d '{"sso": "pending"}'
# ... "role": "developer", "sso": {"status": "pending"}
```

Her next SSO sign-in, with a **verified** email matching hers, is bound to her account - once:

```json
{"name": "carol", "role": "owner", "email": "carol@example.com", "created_by": "cli",
 "sso": {"status": "linked", "issuer": "https://idp.example.com/realms/acme"}}
```

She's an **owner** now: linked, her role is her groups' (she's in `qag-owners`), not what she had. Changing it in QueryAPIGate is refused - `409 sso_managed`, *their role comes from their groups at the identity provider - change it there*. Her email and whether she's active can still change here. `{"sso": null}` unlinks her.

Link people before they first sign in through SSO; someone who signs in first gets an administrator of their own (`carol-2`, if the name is taken), and the two have to be sorted out by hand.

Step 6: SSO only

With every person linked, stop personal tokens routing around the provider's MFA and policies:

```shell
QUERYAPIGATE_SSO_ONLY=yes
```

carol's old token now gets `401`, and issuing her one is refused - `409 sso_only`. Accounts for automation (`ci-deploy`), linked to no one, keep their tokens. The break-glass key keeps working, audited and alerted: it's how owners get in if the provider is down.

When the provider is down

Sign-in through it stops; sessions already open keep working. The Alerts screen says so - `sso_unreachable`, critical: *Console sign-in through SSO is down*. Owners sign in with a token or the break-glass key until it's back.

Good to know

- **Someone leaving** is removed at the provider: they can't sign in again, but a session already open lasts until it expires - `QUERYAPIGATE_SSO_SESSION_HOURS`, default 8. Switching them off in QueryAPIGate ends it at once.
- **Who controls `qag-owners` controls QueryAPIGate.** Keep that group small, and its membership managed like any other privileged group.
- **A default role** for providers without groups: `QUERYAPIGATE_SSO_DEFAULT_ROLE=auditor` with `QUERYAPIGATE_SSO_ALLOWED_DOMAINS=example.com` - for people with a verified email there. Never `owner`; startup refuses one without the other.
- **Linking trusts `email_verified`**: only link at a provider where people can't set and verify an email they don't own.
- **Refused sign-ins** carry a code: `sso_no_role`, `sso_state` (expired, used, or started in another browser - sign in again), `sso_id_token` (the provider's answer didn't verify - check the clock and the client ID), `sso_inactive`, `sso_denied`, `sso_unreachable`, `sso_ambiguous` (two administrators wait for the same email), `sso_busy`.

Next steps

- Give your team their own admin access - administrators, roles and tokens, which this builds on.
- Let AI clients sign in through your identity provider - the same provider, for agents.
- API reference: Console sign-in through SSO.